1. Retryable 사용법

> `@Retryable`은 **spring-context** 모듈에 있다. Spring Boot 4.x는 Spring Framework 7 기반이며, `spring-boot` 모듈이 `spring-context`에 compile 스코프로 의존한다. 그래서 **추가할 의존성이 없다.** 예전처럼 `spring-retry`나 AspectJ를 따로 넣을 필요도 없다.

`@Retryable`은 **따를지 무시할지 선택할 수 있는 메타데이터**다. 그래서 처리를 켜주지 않으면 어노테이션이 붙어 있어도 아무 일도 일어나지 않는다. 에러도 경고도 없이 그냥 한 번 호출하고 끝나므로, 재시도가 안 된다면 가장 먼저 확인할 부분이다.

```
@Configuration
@EnableResilientMethods // @Retryable과
@ConcurrencyLimit 처리를 활성화한다
public class ResilienceConfig {
}
```

또는 `RetryAnnotationBeanPostProcessor` 빈을 직접 등록해서 `@Retryable`만 켤 수도 있다. (`@ConcurrencyLimit`만 켜려면 `ConcurrencyLimitBeanPostProcessor`를 등록한다)

@EnableResilientMethods 속성

속성	기본값	설명
`proxyTargetClass`	`false`	`true` 면 인터페이스 기반 JDK 프록시 대신 **CGLIB(서브클래스) 프록시**를 만든다. `RetryAnnotationBeanPostProcessor`와 `ConcurrencyLimitBeanPostProcessor`에만 영향을 준다

속성	기본값	설명
`order`	`Ordered.LOWEST_PRECEDENCE - 1` (`Integer.MAX_VALUE - 1`)	두 post-processor의 적용 순서다. 기본값은 `@EnableAsync`를 제외한 모든 일반 post-processor **뒤에** 실행되도록 잡혀 있다

`proxyTargetClass`는 공식 문서에서도 여기서 따로 설정하기보다 전역 프록시 설정을 따르고, 특정 빈만 `@Proxyable`로 지정하는 쪽을 권장한다. `order`는 주로 `@Async`와의 감싸는 순서를 바꿀 때 쓴다.

---

1. 기본 사용

```
@Component
public class NotificationClient {

    @Retryable
    public void send(String message) {
        notificationApi.send(message);
    }
}
```

기본 동작

- **모든 예외**에 대해 재시도한다. (checked 예외 포함)
- 최초 실패 후 **최대 3번** 재시도한다. 즉, 총 최대 **4번** 호출한다.
- 시도 사이에 **1초 고정 대기**를 한다. (`delay = 1000`, `multiplier = 1.0`)
- 끝까지 실패하면 **마지막 원본 예외**가 호출자에게 그대로 전파된다. 래핑된 예외가 아니라서 기존 예외 처리 코드를 그대로 쓸 수 있다.

선언 위치

- **메소드 레벨** : 그 메소드에만 적용된다.
- **타입(클래스) 레벨** : 그 클래스 계층에서 **프록시를 거쳐 호출되는** 메소드 전체에 적용된다.

```
@Component
@Retryable(maxRetries = 1)
public class InventoryClient {
    // 이 클래스에서 프록시를 거쳐 호출되는 메소드 전체가 대상이다
}
```

---

2. 속성

속성	기본값	설명
`value`	`{}`	`includes`의 별칭이다. `@Retryable(MyException.class)`처럼 쓴다
`includes`	`{}`	재시도할 예외 타입이다. 비어 있으면 모든 예외가 대상이며, **중첩된 원인(cause)까지** 매칭한다
`excludes`	`{}`	재시도하지 않을 예외 타입이다. 마찬가지로 원인까지 매칭한다
`predicate`	모든 예외 재시도	사용자 정의 판단 로직이다. `includes` / `excludes` **다음에** 적용된다
`maxRetries`	`3`	최대 재시도 횟수다. **총 시도 횟수 = 1 + maxRetries**
`timeout`	`0` (제한 없음)	최초 호출부터 모든 재시도와 대기 시간까지 합친 최대 경과 시간이다 (7.0.2+)
`delay`	`1000`	최초 호출 후 첫 재시도 전 기본 대기 시간이다. `multiplier`가 있으면 곱하기 시작 값이 된다
`jitter`	`0` (없음)	계산된 대기 시간에 무작위 값을 더하거나 뺀다
`multiplier`	`1.0`	다음 대기 = 이전 대기 × multiplier. 기본값 1.0이면 고정 대기다
`maxDelay`	무제한	대기 시간의 상한이다. `jitter`와 `multiplier`가 대기를 늘릴 수 있는 한계를 정한다
`timeUnit`	`MILLISECONDS`	`delay`, `jitter`, `maxDelay`, `timeout`의 시간 단위다

> 시간 관련 속성의 기본 단위는 **밀리초**다. `delay = 2`는 2초가 아니라 2ms다.

---

3. 어떤 예외를 재시도할지 정하기

includes / excludes

```
@Retryable(
        includes = PaymentApiException.class,   // 이 예외만 재시도
        excludes = InvalidCardException.class   // 이 예외는 재시도하지 않음
)
public void something() { ... }
```

`@Retryable(PaymentApiException.class)`는 `@Retryable(includes = PaymentApiException.class)`의 축약형이다.

원인까지 매칭하기 때문에, `PaymentApiException`이 다른 예외로 감싸져서 올라와도 재시도 대상이 된다. 예를 들어 `RuntimeException(cause = PaymentApiException)`도 매칭된다.

> 기본값은 **모든 예외**를 재시도한다. 잘못된 카드 번호처럼 몇 번을 다시 해도 똑같이 실패하는 예외까지 재시도하면 대기 시간만 낭비된다. 네트워크 오류나 일시적인 서버 오류처럼 **다시 하면 성공할 수 있는 예외**만 `includes`로 좁히는 것이 좋다.

predicate : 메시지 등으로 세밀하게 판단

예외 타입만으로 구분이 안 될 때 쓴다. 같은 예외 타입인데 메시지나 에러 코드에 따라 재시도 여부가 달라지는 경우다.

```
public class TemporaryErrorPredicate implements
MethodRetryPredicate {

    @Override
    public boolean shouldRetry(Method method, Throwable
throwable) {
        return throwable.getMessage() != null
                &&
throwable.getMessage().contains("temporary");
    }
}

@Retryable(predicate = TemporaryErrorPredicate.class)
public void execute() { ... }
```

- 구현체는 **메소드마다 하나씩** 인스턴스화되며, 생성자 주입이나 `@Autowired`로 다른 빈을 쓸 수 있다.
- `includes` / `excludes`와 같이 쓸 수 있지만, 항상 **그 다음에** 적용된다. 즉, `includes`를 통과한 예외에 대해서만 predicate가 호출된다.

---

4. 대기 시간 : 백오프 조절

```
@Retryable(
        includes = PaymentApiException.class,
        maxRetries = 4,
        delay = 100,
        jitter = 10,
        multiplier = 2,
        maxDelay = 1000
)
```

```
public Payment pay(String orderId) {
    ...
}
```

위 설정으로 실제 대기 시간이 어떻게 늘어나는지 계산하면 다음과 같다.

시도	대기 (jitter 제외)	jitter 범위
최초 호출	-	-
1번째 재시도 전	100ms	±10ms
2번째 재시도 전	200ms	±20ms
3번째 재시도 전	400ms	±40ms
4번째 재시도 전	800ms	±80ms

총 5번 호출되고, 다음 대기가 1600ms가 되더라도 `maxDelay = 1000`에 막힌다.

jitter는 계산된 대기에 무작위로 더하거나 빼서 `delay - jitter` ~ `delay + jitter` 사이의 값을 만든다. **단, 기본 `delay`보다 작아지거나 `maxDelay`보다 커지지 않는다.** `multiplier`는 대기와 jitter **모두에** 곱해진다.

`delay = 0`에 양수 jitter를 주면 `multiplier`와 상관없이 대기가 늘지 않고, 매번 `0` ~ `min(jitter, maxDelay)` 사이의 무작위 값이 된다.

> **jitter가 필요한 이유** : 서버 장애로 여러 클라이언트가 동시에 실패하면, jitter 없이는 모두 같은 간격으로 동시에 재시도해서 복구 중인 서버를 다시 몰아친다(thundering herd). 대기에 무작위 값을 섞으면 재시도 시점이 흩어진다.

---

5. 설정 파일로 값 빼기

```
payment.retry.max-retries=5
payment.retry.delay=500ms
```

```
@Retryable(
        includes = PaymentApiException.class,
        maxRetriesString = "${payment.retry.max-
retries}",
        delayString = "${payment.retry.delay}"
)
public PaymentResult pay(String orderId) {
    ...
}
```

- `...String` 속성은 `${}` 플레이스홀더와 SpEL을 지원한다.
- 값이 비어 있지 않으면 **숫자 속성보다 우선**한다.
- 시간 속성 (`delayString`, `jitterString`, `maxDelayString`, `timeoutStri ng`)은 여러 형식을 받는다.

형식	예시	해석
정수	`500`	`timeUnit` 기준 (기본 밀리초)
SIMPLE	`500ms`, `2s`, `1m`	단위를 직접 표기
ISO-8601	`PT0.5S`, `PT2S`	`java.time.Duration` 형식

환경별로 재시도 정책을 다르게 가져가야 할 때(예: 로컬은 빠르게 실패, 운영은 여유 있게 재시도) 유용하다.

---

6. 전체 시간 제한 : timeout

```
@Retryable(maxRetries = 100, delay = 50, timeout =
5000) // 최대 5초
public void call() { ... }

// 단위를 바꾸려면 timeUnit을 쓴다
@Retryable(maxRetries = 100, delay = 50, timeout = 5,
timeUnit = TimeUnit.SECONDS)
public void callInSeconds() { ... }
```

`timeout`은 **최초 호출부터 모든 재시도와 대기 시간까지 합친 최대 경과 시간**이다. 기본값은 `0`으로 제한이 없다. (Spring Framework 7.0.2부터 지원)

`maxRetries`만으로는 전체 소요 시간을 예측하기 어렵다. 호출 하나가 얼마나 걸릴지 모르기 때문이다. 사용자 요청을 처리하는 중이라 응답 시간이 정해져 있다면, `maxRetries`는 넉넉히 두고 `timeout`으로 상한을 거는 편이 안전하다.

> `timeout`도 기본 단위가 밀리초다. `timeout = 120`은 120초가 아니라 **120ms**다.

---

7. 주의사항

같은 클래스 안에서 호출하면 재시도가 안 된다

- `@Retryable`은 **프록시를 거쳐 호출될 때만** 동작한다. `this.send()`처럼 자기 자신의 메소드를 부르면 프록시를 거치지 않는다.
- 재시도할 메소드를 다른 빈으로 분리하거나, 해당 부분만 `RetryTemplate`으로 감싼다.

```
@Component
public class OrderService {

    public void order() {
        send(); // 프록시를 거치지 않으므로 재시도되지 않는다
    }

    @Retryable
    public void send() { ... }
}
```

메소드 전체가 다시 실행된다

- 재시도는 메소드를 처음부터 다시 호출한다. 부수 효과가 있는 코드(DB 저장, 메시지 발행 등)와 재시도할 코드가 한 메소드에 섞여 있으면, 부수 효과까지 여러 번 실행된다.
- 이 경우 `RetryTemplate`으로 일부만 감싸거나 메소드를 나눠야 한다.

중간 예외는 호출자에게 보이지 않는다

- 호출자는 마지막 예외만 받는다. 중간에 어떤 예외로 몇 번 실패했는지 로깅하려면, Spring이 매 실패마다 발행하는 `MethodRetryEvent`를 `@EventListener`로 받으면 된다.

import 충돌

- RetryTemplate용 함수형 인터페이스 `org.springframework.core.retry.Retryable`과 이름이 같다. 한 파일에서 둘 다 써야 하면 한쪽은 풀네임으로 적어야 한다.